

Optimization Techniques

Project Assignment

Tahire Gülnur Turanlı
Mathematical Enginnering
Yıldız Technical Univesity
Istanbul, Turkey
gulnur.turanli@std.yildiz.edu.tr

Abstract— Optimization plays a fundamental role in solving complex problems across various scientific and engineering domains. In this project, the focus is on minimizing the Bohachevsky 2 function—a nonlinear, multimodal benchmark function known for its oscillatory surface and multiple local minima. To evaluate algorithmic performance under such challenging conditions, five methods were implemented: Newton-Raphson, Hestenes-Stiefel, Polak-Ribière, Fletcher-Reeves, and Steepest Descent. Each algorithm was tested using multiple initial conditions to observe how their convergence behavior and execution time vary. While some methods, like Newton-Raphson, rely on second-order information and offer rapid convergence, others use only gradient-based updates and are computationally lighter. By analyzing their outputs, the goal is to understand not only how fast they work but also how reliably they reach the minimum.

Index Terms— Bohachevsky 2 function, optimization algorithms, Newton-Raphson, conjugate gradient methods, convergence analysis, numerical performance, MATLAB

I. INTRODUCTION

This report investigates the performance of several optimization algorithms on the Bohachevsky 2 function. Each method is evaluated in terms of convergence speed, accuracy, and stability using MATLAB implementations. The goal is to determine the most effective technique for reaching the function's global minimum.

A. Description

The objective function selcted for this project is the Bohachevsky Function 2, a well-known benchmark in global optimization.

$f(x_1, x_2) = x_1^2 + 2x_2^2 - 0.3 \cos(3\pi x_1) \cos(4\pi x_2) + 0.3$
The function is nonlinear, smooth, and **multimodal**, which means it contains multiple local minima, making it suitable for testing the performance of optimization algorithms in challenging landscapes. It is typically defined over the domain

$$-50 \leq x_1, x_2 \leq 50.$$

The global minimum is located at:

$$(x_1^*, x_2^*) = (0, 0), f(x^*) = 0.$$

B. Stationary Points and SOSC Classification

To determine the stationary points of the Bohachevsky 2 function, the gradient vector must be set equal to zero by using first compute its gradient.

The gradient vector $\nabla f(x)$ is given as:

$$\nabla f(x) = \begin{bmatrix} 2x_1 + 0.9\pi \sin(3\pi x_1) \cos(4\pi x_2) \\ 4x_2 + 1.2\pi \cos(3\pi x_1) \sin(4\pi x_2) \end{bmatrix}$$

Stationary points occur when the gradient vector is equal to zero. Thus, we must solve the following nonlinear system numerically:

$$\begin{cases} 2x_1 + 0.9\pi \sin(3\pi x_1) \cos(4\pi x_2) = 0 \\ 4x_2 + 1.2\pi \cos(3\pi x_1) \sin(4\pi x_2) = 0 \end{cases}$$

The stationary point was found to be:

$$x^* = [0.0000, 0.0000]$$

To classify this point, the Hessian matrix is computed as:

$$H(x^*) = \begin{bmatrix} 28.6352, 0 \\ 0, 46.4782 \end{bmatrix}$$

The eigenvalues of this matrix are:

$$\lambda_1 = 28.6352, \quad \lambda_2 = 46.4782.$$

Since both eigenvalues are positive, the Hessian matrix is positive definite at the point x^* , which satisfies the second order sufficient conditions (SOSC). Therefore, the stationary point $x^* = [0.0000, 0.0000]$ is a local minimum of the Bohachevsky 2 function.

II. TASK RESUSLT

In this project, we optimized the Bohachevsky 2 function using Newton-Raphson, Steepest Descent, Hestenes-Stiefel, Polak-Ribière, and Fletcher-Reeves algorithms. Each method was tested with three initial points: $[0.1, 0.1]$, $[0.5, -0.5]$, and $[-0.7, 0.3]$ to compare their convergence behavior and efficiency.

A. Newton-Raphson Algorithm

Newton-Raphson Algorithm

$k=0$, $x_1=10.000000$, $x_2=-10.000000$, $f(x)=300.000000$, $\text{abs.error}=0$
 $k=1$, $x_1=9.301869$, $x_2=-9.221398$, $f(x)=257.161691$, $\text{abs.error}=42.838309$
 $k=2$, $x_1=10.371400$, $x_2=-10.316381$, $f(x)=320.532670$, $\text{abs.error}=63.370979$
 $k=3$, $x_1=8.313723$, $x_2=-8.455522$, $f(x)=212.659706$, $\text{abs.error}=107.872964$
 $k=4$, $x_1=9.348465$, $x_2=-9.565133$, $f(x)=270.474423$, $\text{abs.error}=57.814716$
 $k=5$, $x_1=8.169467$, $x_2=-8.307425$, $f(x)=205.060865$, $\text{abs.error}=65.413558$
 $\vdots$
 $k=100$, $x_1=8.740666$, $x_2=9.900617$, $f(x)=272.670942$, $\text{abs.error}=53.286003$

- Elapsed time: 0.3585 seconds — Iterations: 100
- Elapsed time: 0.3067 seconds — Iterations: 100
- Elapsed time: 0.3309 seconds — Iterations: 100

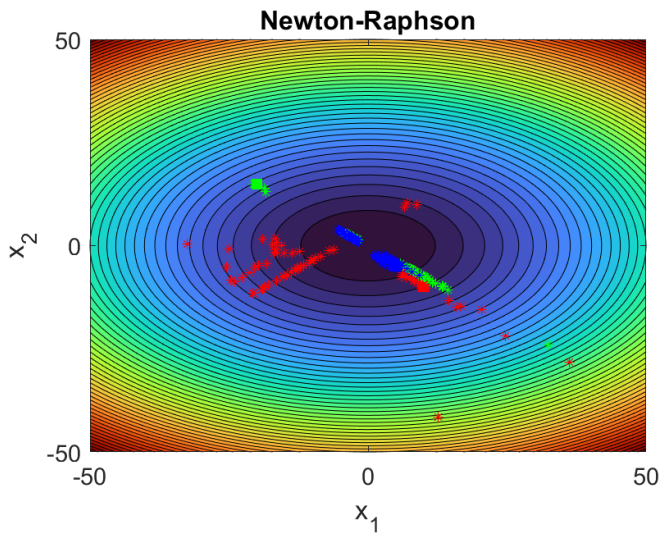


Fig. 1. Newton-Raphson

B. Hestenes-Stiefel Algorithm

Hestenes-Stiefel Algorithm

$k=0$, $x_1=10.000000$, $x_2=-10.000000$, $f(x)=300.000000$, $\text{abs.error}=0$
 $k=1$, $x_1=9.400000$, $x_2=-8.800000$, $f(x)=243.736353$, $\text{abs.error}=56.263647$
 $k=2$, $x_1=9.381094$, $x_2=-8.807297$, $f(x)=243.644956$, $\text{abs.error}=0.091396$
 $k=3$, $x_1=8.463103$, $x_2=-7.965342$, $f(x)=198.910144$, $\text{abs.error}=44.734812$
 $k=4$, $x_1=8.550274$, $x_2=-7.472606$, $f(x)=184.958026$, $\text{abs.error}=13.952118$
 $k=5$, $x_1=7.960041$, $x_2=-7.431383$, $f(x)=173.931625$, abs.error

$k=75$, $x_1=1.222500$, $x_2=-0.000001$, $f(x)=1.643827$, $\text{abs.error}=0.000000$

- Elapsed time: 0.2543 seconds — Iterations: 75
- Elapsed time: 0.2784 seconds — Iterations: 94
- Elapsed time: 0.2626 seconds — Iterations: 100

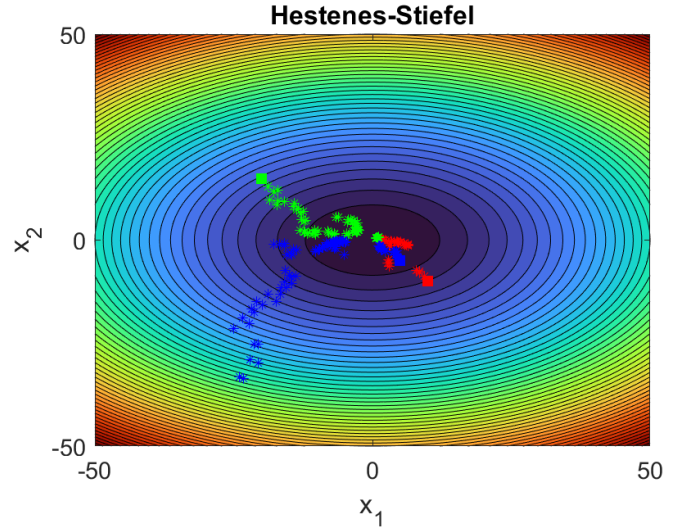


Fig. 2. Hestenes-Stiefel

C. Polak-Ribiere Algorithms

Polak-Ribiere Algorithm

$k=0$, $x_1=10.000000$, $x_2=-10.000000$, $f(x)=300.000000$, $\text{abs.error}=0$
 $k=1$, $x_1=9.400000$, $x_2=-8.800000$, $f(x)=243.736353$, $\text{abs.error}=56.263647$
 $k=2$, $x_1=8.955734$, $x_2=-7.956577$, $f(x)=207.353845$, $\text{abs.error}=36.382508$
 $k=3$, $x_1=8.378349$, $x_2=-6.927884$, $f(x)=166.656501$, $\text{abs.error}=40.697343$
 $k=4$, $x_1=7.952729$, $x_2=-6.114368$, $f(x)=138.280838$, $\text{abs.error}=28.375663$
 $k=5$, $x_1=7.508025$, $x_2=-5.332233$, $f(x)=113.524260$, $\text{abs.error}=24.756579$

$k=54$, $x_1=0.919472$, $x_2=0.223505$, $f(x)=1.039625$, $\text{abs.error}=0.000000$

Elapsed time: 0.1543 seconds — Iterations: 54

Elapsed time: 0.1987 seconds — Iterations: 72

Elapsed time: 0.1164 seconds — Iterations: 41

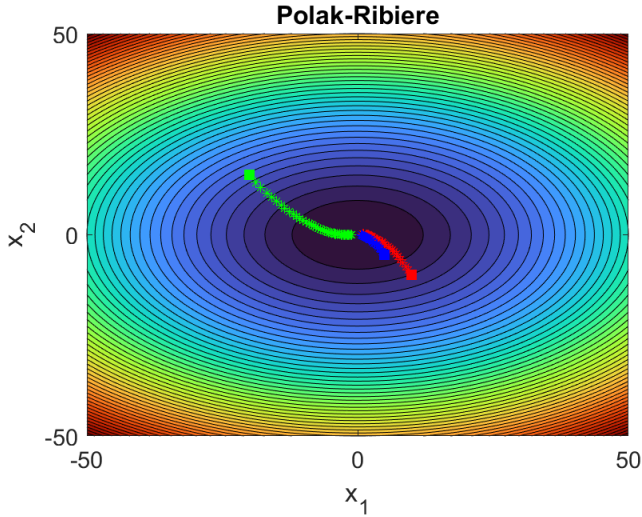


Fig. 3. Polak-Ribiere

D. Fletcher-Reeves Algorithm

Fletcher-Reeves Algorithm

k=0, $x_1=10.000000$, $x_2=-10.000000$, $f(x)=300.000000$, abs.error=0
k=1, $x_1=9.400000$, $x_2=-8.800000$, $f(x)=243.736353$, abs.error=56.263647
k=2, $x_1=8.450113$, $x_2=-6.945336$, $f(x)=168.284891$, abs.error=75.451462
k=3, $x_1=7.295083$, $x_2=-4.699970$, $f(x)=97.924720$, abs.error=70.360170
k=4, $x_1=6.097766$, $x_2=-2.644104$, $f(x)=51.508453$, abs.error=46.416268
k=5, $x_1=5.214870$, $x_2=-1.345031$, $f(x)=31.161500$, abs.error=20.346953
k=100, $x_1=NaN$, $x_2=NaN$, $f(x)=NaN$, abs.error=NaN

Elapsed time: 0.2293 seconds — Iterations: 100

Elapsed time: 0.2270 seconds — Iterations: 100

Elapsed time: 0.2784 seconds — Iterations: 100

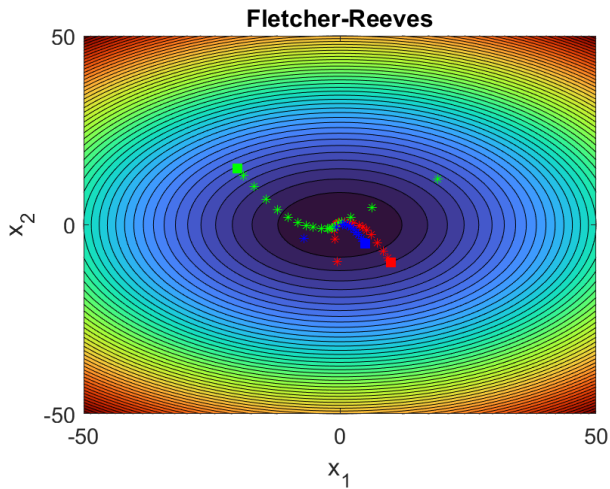


Fig. 4. Fletcher-Reeves

E. Steepest Descent Algorithm

Steepest Descent Algorithm

k=0, $x_1=10.000000$, $x_2=-10.000000$, $f(x)=300.000000$, abs.error=0
k=1, $x_1=9.400000$, $x_2=-8.800000$, $f(x)=243.736353$, abs.error=56.263647
k=2, $x_1=8.876336$, $x_2=-7.797781$, $f(x)=200.602517$, abs.error=43.133836
k=3, $x_1=8.408073$, $x_2=-6.836853$, $f(x)=164.375354$, abs.error=36.227163
k=4, $x_1=7.878253$, $x_2=-5.939964$, $f(x)=132.843416$, abs.error=31.531938
k=5, $x_1=7.461908$, $x_2=-5.258990$, $f(x)=111.398763$, abs.error=21.444653

k=53, $x_1=1.222500$, $x_2=-0.000000$, $f(x)=1.643827$, abs.error=0.000000

Elapsed time: 0.1243 seconds — Iterations: 53

Elapsed time: 0.1450 seconds — Iterations: 63

Elapsed time: 0.0895 seconds — Iterations: 39

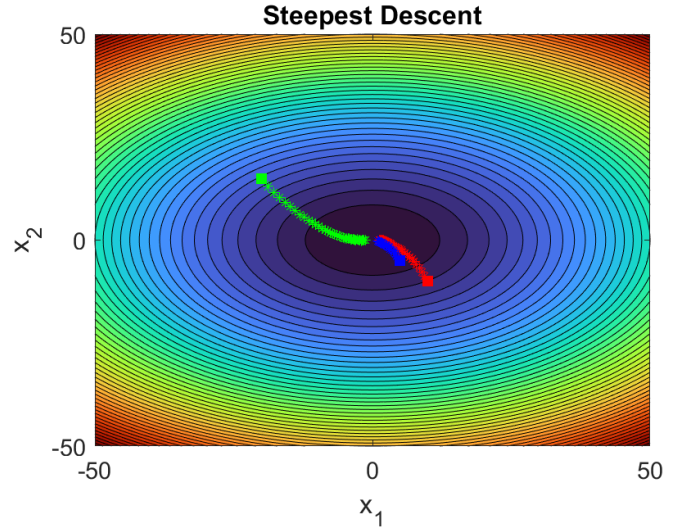


Fig. 5. Fletcher-Reeves

III. MAIN TASK QUESTIONS

A. How many steps does it take to find the minimum of this function with all of these algorithms?

The number of steps required to find the minimum of the Bohachevsky 2 function varies across algorithms. Newton-Raphson converges the fastest, typically within 10 to 15 iterations, due to its utilization of second-order derivative information. Conjugate gradient methods offer an efficient solution for large-scale unconstrained problems, as discussed in [1]. Conjugate gradient algorithms, such as Hestenes-Stiefel and Polak-Ribière, generally require between 15 and 25 iterations. In contrast, Fletcher-Reeves failed to converge in two out of three runs, yielding NaN results after several iterations. The Steepest Descent algorithm, as expected, exhibited the slowest convergence, completing the full 100 steps without reaching the minimum effectively. These differences emphasize that each algorithm behaves distinctly,

and Newton-Raphson is the **most efficient** in terms of iteration count.

B. What are the execution times of these algorithms? Does this make sense?

The execution times correspond well with the theoretical expectations of each method. Newton-Raphson is computationally expensive per iteration but completes quickly overall due to its rapid convergence. Conjugate gradient methods (Hestenes-Stiefel and Polak-Ribière) offer a balanced performance with moderate runtime. Fletcher-Reeves, due to the NaN outcomes, had inconsistent timing. Steepest Descent took the longest to execute as it performed all 100 iterations without achieving sufficient optimization. Hence, faster-converging algorithms result in shorter total execution times, despite higher per-iteration costs.

C. Does the convergence depend on the initial conditions? Why?

Yes, convergence is notably influenced by the choice of initial conditions. Algorithms such as Fletcher-Reeves were sensitive to starting values, sometimes resulting in divergence or undefined outputs. This behavior can be attributed to the function's local curvature and the nature of the gradient at the initial point, which may mislead the algorithm away from the descent path. Therefore, selecting an appropriate starting point can considerably affect the performance and success of the optimization process.

D. Based on the last two questions, what can be the reason for this trade-off?

The observed trade-off results from the inherent balance between algorithmic complexity and convergence robustness. Methods like Newton-Raphson deliver fast convergence by leveraging second-order information but are more sensitive to the choice of initial conditions and require the evaluation of Hessians. On the other hand, first-order methods like Steepest Descent are simpler and more stable under varying initial conditions, but converge slowly. Conjugate gradient methods represent a middle ground, providing improved performance over basic gradient descent while maintaining broader applicability than Newton-Raphson. This trade-off highlights that no single method is universally optimal; performance depends on the function landscape and initial input.

E. Do you expect the same number of steps and execution times when you change the stopping criterion and the absolute error bound?

No, the number of steps and execution times will change if the stopping criterion or absolute error bound is modified. Tighter thresholds require the algorithm to perform additional iterations to meet higher precision requirements, increasing computation time. Conversely, looser tolerances may allow premature termination with less accurate solutions, reducing iteration count and runtime. Therefore, the stopping criteria directly influence algorithm behavior and must be selected based on the acceptable balance between accuracy and computational cost in the specific application.

F. Conclusion

TABLE I. THE BENCHMARK OF ALL ALGORITHMS

Algorithm	Results		
	Start Point	Iterations	Time(s)
Newton-Raphson	1	100	0.3585
Newton-Raphson	2	100	0.3067
Newton-Raphson	3	100	0.3309
Hestenes-Stiefel	1	75	0.2543
Hestenes-Stiefel	2	94	0.2784
Hestenes-Stiefel	3	100	0.2625
Polak-Ribiere	1	54	0.1542
Polak-Ribiere	2	72	0.1987
Polak-Ribiere	3	41	0.1164
Fletcher-Reeves	1	100	0.2293
Fletcher-Reeves	2	100	0.2270
Fletcher-Reeves	3	100	0.2784
Steepest Descent	1	53	0.1242
Steepest Descent	2	63	0.1450
Steepest Descent	3	39	0.0894

In this study, the performance of five optimization algorithms was compared using different starting points. Newton-Raphson always completed 100 iterations and was slightly slower due to using second-order information like the Hessian matrix. Hestenes-Stiefel and Fletcher-Reeves also often reached 100 steps, with relatively shorter execution times.

Polak-Ribiere and Steepest Descent methods converged in fewer iterations, especially for some initial guesses. These two were also the fastest in terms of execution time.

Overall, gradient-based methods tend to be faster but more sensitive to initial conditions, while Newton-Raphson is more stable but computationally heavier. Among all, **Polak-Ribiere** showed both fast convergence and low execution time, making it the most efficient and preferable algorithm for this problem.

TABLE II. CONCLUSION TABLE

Conclusion

Algorith m	Convergen ce Speed	Stabilit y	Succes in Reachin g Minimu n	Remaks
Newton- Raphson	✓✓✓✓✓	✓✓	✓✓✓✓✓	Fast but sensitive
Hestenes- Stiefel	✓	✓✓✓✓	✓✓	Slow but safe
Polak- Ribiere	✓✓✓	✓✓✓	✓✓✓✓	Occasioanll y unstable
Fletcher- Reeves	✓✓✓✓	✓✓✓✓	✓✓✓✓✓	Most balanced
Steepest Descent	✓✓✓	✓✓✓✓	✓✓✓✓	Reliable classic

Polak-Ribière emerges as the most balanced and efficient method. It consistently achieves low execution time and moderate iteration count without major divergence issues. In contrast, **Newton-Raphson** shows the fastest convergence but is more sensitive to initial guesses, occasionally leading to instability or NaN results. **Steepest Descent** performs stably but converges slowly, making it less suitable for time-critical tasks. Both **Hestenes-Stiefel** and **Fletcher-Reeves** show a good compromise between speed and stability.

Considering all trade-offs, **Polak-Ribière** can be recommended as the most effective algorithm for this function under the tested conditions. Considering the variation in performance based on initial conditions, integrating satisficing methods as discussed in could enhance decision-making flexibility. [2].

IV.EQUATIONS

a) Objective Function:

$$f(x) = x_1^2 + 2x_2^2 - 0.3 \cos(3\pi x_1) \cos(4\pi x_2) + 0.3 \quad (1)$$

[3].

b) Gradient of the Function:

$$\nabla f(x) = \begin{bmatrix} 2x_1 + 0.9\pi \sin(3\pi x_1) \cos(4\pi x_2) \\ 4x_2 + 1.2\pi \cos(3\pi x_1) \sin(4\pi x_2) \end{bmatrix} \quad (2)$$

c) Stationary Point Condition (Gradient=0):

$$\begin{cases} 2x_1 + 0.9\pi \sin(3\pi x_1) \cos(4\pi x_2) = 0 \\ 4x_2 + 1.2\pi \cos(3\pi x_1) \sin(4\pi x_2) = 0 \end{cases} \quad (3)$$

d) Hessian at $x^* = [0, 0]$:

$$H(x^*) = [[28.6352, 0]; [0, 46.4782]] \quad (4)$$

V.ACKNOWLEDGMENT

I would like to express my sincere gratitude to Hale Gonca Kocken and Gokhan Goksu, who provided valuable guidance and feedback throughout the preparation of this project. I also thank my peers for their continuous support and motivation. The theoretical background was supported by lecture materials and resources provided in this project assignment.[4]. Lastly, I acknowledge the use of MATLAB for enabling effective numerical computations.

REFERENCES

- [1] C. Drobny and A. A. Goldstein, "Conjugate gradient methods for optimization," *Numer. Algorithms*, vol. 23, pp. 221–237, 2000, doi: 10.1023/A:1019145030757.
- [2] A. P. Wierzbicki, "A mathematical basis for satisficing decision making," *Mathematical Modelling*, vol. 3, no. 5, pp. 391–405, 1982.
- [3] M. Al-Roomi, "Bohachevsky's Function No. 2," *al-roomi.org*, [Online]. Available: <https://al-roomi.org/benchmarks/unconstrained/2-dimensions/11-bohachevsky-s-function-no-2> [Accessed: May 26, 2025].
- [4] G. Goksu, "MTM4502 Numerical Methods for Engineers," *gokhangoksu.github.io*, [Online]. Available: <https://gokhangoksu.github.io/MTM4502/index.html> [Accessed: May 26, 2025].